

```
-T ==> set image type
-C ==> set compression type
-a ==> set load address (hex)
-e ==> set entry point (hex)
-n ==> set image name
-d ==> use image data from file
```

The first command converts the ELF into a raw binary. This binary is then passed to *mkimage*, which is a utility to generate the u-boot specific kernel image. *mkimage* is the utility provided by u-boot. The generated kernel image is named *uImage*.

The Linux kernel build system

One of the beautiful things about the Linux kernel is that it is highly configurable and the same code base can be used for a variety of applications, ranging from high end servers to tiny embedded devices. And the infrastructure, which plays an important role in achieving this in an efficient manner, is the kernel build system, also known as *kbuild*. The kernel build system has two main components – makefile and Kconfig.

Makefile: Every sub-directory has its own makefile, which is used to compile the files in that directory and generate the object code out of that. The top level makefile percolates recursively into its sub-directories and invokes the corresponding makefile to build the modules and finally, the Linux kernel image. The makefile builds only the files for which the configuration option is enabled through the configuration tool.

Kconfig: As with the makefile, every sub-directory has a Kconfig file. Kconfig is in configuration language and Kconfig files located inside each sub-directory are the programs. Kconfig contains the entries, which are read by configuration targets such as *make menuconfig* to show a menu-like structure.

So we have covered makefile and Kconfig and at present they seem to be pretty much disconnected. For *kbuild* to work properly, there has to be some link between the Kconfig and makefile. And that link is nothing but the configuration symbols, which generally have a prefix *CONFIG_*. These symbols are generated by a configuration target such as *menuconfig*, based on entries defined in the Kconfig file. And based on what the user has selected in the menu, these symbols can have the values 'y', 'n', or 'm'.

Now, as most of us are aware, Linux supports hot plugging of the drivers, which means, we can dynamically add and remove the drivers from the running kernel. The drivers which can be added/removed dynamically are known as modules. However, drivers that are part of the kernel image can't be removed dynamically. So, there are two ways to have a driver in the kernel. One is to build it as a part of the kernel, and the other is to build it separately as a module for hot-plugging. The value 'y' for *CONFIG_*, means the corresponding driver will be part of the kernel image; the value 'm' means it will be built as a module and value 'n'

means it won't be built at all. Where are these values stored? There is a file called *.config* in the top level directory, which holds these values. So, the *.config* file is the output of the configuration target such as *menuconfig*.

Where are these symbols used? In makefile, as shown below:

```
obj-$(CONFIG_MYDRIVER) += my_driver.o
```

So, if *CONFIG_MYDRIVER* is set to value 'y', the driver *my_driver.c* will be built as part of the kernel image and if set to value 'm', it will be built as a module with the extension *.ko*. And, for value 'n', it won't be compiled at all.

As you now know a little more about *kbuild*, let's consider adding a simple character driver to the kernel tree.

The first step is to write a driver and place it at the correct location. I have a file named *my_driver.c*. Since it's a character driver, I will prefer adding it at the *drivers/char/* sub-directory. So copy this at the location *drivers/char/* in the kernel.

The next step is to add a configuration entry in the *drivers/char/Kconfig* file. Each entry can be of type *bool*, *tristate*, *int*, *string* or *hex*. *bool* means that the configuration symbol can have the values 'y' or 'n', while *tristate* means it can have values 'y', 'm' or 'n'. And 'int', 'string' and 'hex' mean that the value can be an integer, string or hexadecimal, respectively. Given below is the segment of code added in *drivers/char/Kconfig*:

```
config MY_DRIVER
    tristate "Demo for My Driver"
    default m
    help
Adding this small driver to kernel for
demonstrating the kbuild
```

The first line defines the configuration symbol. The second specifies the type for the symbol and the text which will be shown as the menu. The third specifies the default value for this symbol and the last two lines are for the help message. Another thing that you will generally find in a Kconfig file is 'depends on'. This is very useful when you want to select the particular feature, only if its dependency is selected. For example, if we are writing a driver for i2c EEPROM, then the menu option for the driver should appear only if the 2c driver is selected. This can be achieved with the 'depends on' entry.

After saving the above changes in Kconfig, execute the following command:

```
$ make menuconfig
```

Now, navigate to *Device Drivers->Character devices* and you will see an entry for *My Driver*.

By default, it is supposed to be built as a module. Once you are done with configuration, exit the menu and save the configuration. This saves the configuration in *.config* file. Now,